

FlowCast

Predicting city traffic 30 minutes before it happens

A machine-learning system for the Northline Corridor · 25 road segments · 5 months of sensor data

Internship Project · Data & AI

THE PROBLEM

Control rooms react. They don't predict.

Signal timings are fixed by clock

They can't respond when congestion builds faster than expected

Incidents are spotted manually

Patrols are positioned after a crash, not before

Weather isn't accounted for

Rain changes flow in ways a fixed plan never anticipates

Planners work on intuition

No single evidence base linking weather, events and flow

The corridor carries roughly 210,000 vehicle trips on a weekday.

THE SOLUTION

One question, four answers, every 30 minutes

Given everything we know up to now — what will each road segment look like in 30 minutes?

Traffic volume

how many vehicles

a number

Average speed

how fast they move

a number

Congestion level

Free-flow to Severe

a category

Accident risk

chance of an incident

a probability

Predicting a number is called regression. Predicting a category is called classification.

THE DATA

Three files, five months, 25 segments

178,468

sensor readings received

25

road segments monitored

30 min

between each reading

traffic_sensor_log.csv

Vehicle counts, speed, occupancy, travel time, incidents — one row per segment per half hour

weather_observations.csv

Temperature, rainfall and visibility from three stations, recorded hourly

calendar_events.csv

One row per day: 6 public holidays, 6 event days, 11 roadwork days

1 January – 31 May 2025. Synthetic data, deliberately made messy to imitate a real sensor feed.

WHAT WAS BROKEN

Real sensor data is never clean

712

impossible readings

negative counts, 300 km/h speeds

1,759

duplicate rows

sensors retransmitting

5,203

missing time slots

absent from the file entirely

13 -> 4

weather spellings

'RAIN', 'rainy', 'Rain'

4,400

blank values per column

sensor dropouts

2

date formats

05/03 vs 2025-03-05

Every fix is logged. Rejected rows are quarantined with a reason, never silently deleted.

THE DANGEROUS ONE

5,203 slots weren't blank. They were absent.

What the code sees

07:00 → 184 vehicles

09:30 → 470 vehicles

"previous reading" = 07:00

The program believes 07:00 was
30 minutes before 09:30.

After rebuilding the timetable

07:00 → 184 vehicles

07:30 → (missing)

08:00 → (missing)

09:00 → (missing)

09:30 → 470 vehicles

Gaps are now visible.

Nothing crashes. No error appears. The numbers just come out quietly wrong.

Fix: rebuild the full grid — 25 segments × 48 windows × 151 days = 181,200 slots — before computing anything.

THE TRAP

One column was secretly the answer

The data has a column called `congestion_level`. It looks like useful information. It isn't — it is just the vehicle count sorted into four buckets.

`vehicles ÷ capacity`
`>100% = Severe`

`<50% = Free-flow`

`50–80% = Moderate`

`80–100% = Heavy`

99.64%

the rule matched existing labels

~99%

fake score if you use it

5 tests

exist purely to prevent it

Give the model the vehicle count and it 'predicts' congestion almost perfectly — because it was handed the answer.

Also found: `vehicle_count` duplicates `traffic_volume` in 97.4% of rows. `travel_time` is simply `distance ÷ speed`.

HOW IT WORKS

93 clues, built by hand

The computer knows nothing about traffic. It doesn't know rush hour exists. Every clue has to be built for it.

Time

hour, weekday, is it rush hour

Recent history

volume 30 min, 1 hr, 2 hrs ago

Weekly memory

same slot yesterday and last week

Rolling averages

mean of the last 4 and 8 slots

Direction

is volume rising or falling?

Weather

raining, low visibility,
temperature

Calendar

holiday, event, roadworks

Combinations

rain DURING rush hour

Good clues matter more than a clever model. Every clue comes from at least 30 minutes before the prediction.

The exam is never seen in advance

TRAINING

8 Jan – 18 Apr

120,950 rows

learn the patterns

VALIDATION

19 Apr – 9 May

25,925 rows

tune the settings

TEST

10 – 31 May

25,925 rows

final score, used once

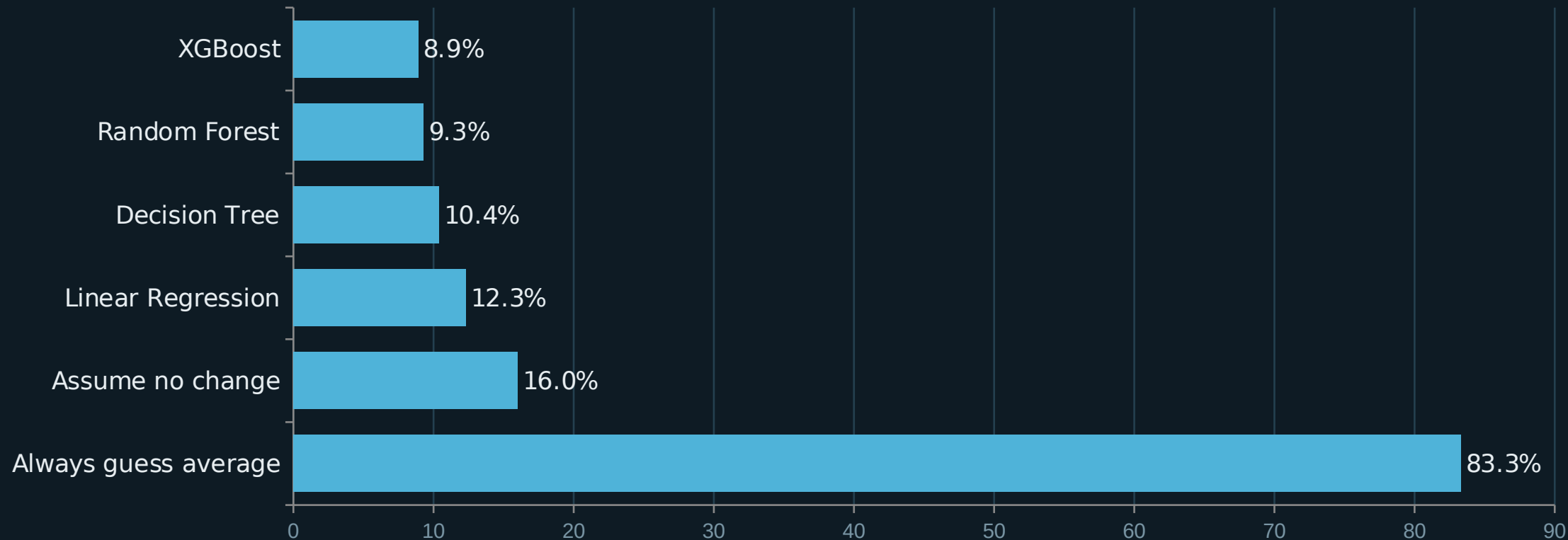
Why split by date and not randomly?

Random splitting would put 3:00pm Tuesday in training and 3:30pm Tuesday in the test. The model would be predicting a gap it has already seen both sides of — impossible in real life, because the future hasn't happened yet.

The final three weeks are hidden completely during learning, then used exactly once.

RESULTS

Six methods, one hidden test set



Lower is better. Tested on 10-31 May 2025 — three weeks the models never saw during training.

THE HEADLINE

8.9% error — and 45% better than free

8.9%

average prediction error

16.0%

error from assuming no change

45%

reduction in error

If 200 vehicles actually pass, the system typically predicts between 182 and 218.

Half an hour before it happens.

Speed

8.3% error

Travel time

9.1% error

Prediction speed

0.02 seconds

Whole corridor forecast in 0.02 seconds against a requirement of 30 seconds.

Two targets missed — for different reasons

Congestion classification

0.77 / target 0.80

A metric problem, not a model problem.

The score weights all four categories equally — but 'Severe' is only 5.5% of slots, so the model sees few examples of it.

Overall accuracy is 87.7%.

Accident risk

0.64 / target 0.75

A data problem, and I proved it.

Accidents occur in only 0.9% of slots — about 1 in 110.

So I ran an experiment to find out whether the fault was mine or the data's →

THE EXPERIMENT

Is it my model, or is it the data?

When a model underperforms there are only two possibilities — and they need opposite responses.

1. The model is bad → try harder

2. The data lacks the answer → stop

So I built a deliberately CHEATING model — allowed to see the exact moment being predicted, which no real forecast could ever have.

0.632

honest model

0.638

cheating model

+0.006

gain from cheating

Cheating bought nothing. The information simply isn't in this data — no amount of tuning reaches 0.75.

USEFUL ANYWAY

A weak model that still saves time

Absolute accuracy is poor. But the ranking still works — and ranking is what a patrol team actually acts on.

2.64x

more incidents found

Patrol only the 10% of road-hours the model ranks riskiest, and you find 2.6 times as many incidents as patrolling at random.

Recommendation: judge this feature on ranking, not on the original metric — and collect road-geometry and collision-history data before promising more.

A 'failed' model can still deliver real operational value if you measure the right thing.

Three things I added

Confidence ranges

Every forecast comes with a range, not just a number. It promises to be right 80% of the time — measured on the hidden weeks, it was right 80.5%. It widens in rush-hour rain and narrows on quiet nights.

A neural network — and a recommendation against it

Scored 55.59 against the simple model's 55.86. Under half a percent better, with 64,269 parameters and 12× the training time. My recommendation is not to deploy it.

Calibrated risk numbers

Rare-event models distort their own output scale — it was reporting 44% risk on a road with a 0.96% rate. A correction step brought the average to 0.80% without changing the ranking at all.

Building something and concluding it isn't worth using is a legitimate engineering result.

Built so someone else can check it

21

automated tests, all passing

5

test the leakage rule alone

1

command rebuilds everything

0.02s

to forecast the whole corridor

What the tests actually protect

- No feature may come from the moment being predicted
- No feature may correlate above 0.99 with the target — that would signal a leak
- Every lag must point exactly 30 minutes back, never across a gap
- The timetable must be complete and evenly spaced
- The hand-written maths must match the library version

A crash announces itself. A silent leak doesn't — so the tests target the failures you can't see.

LIVE

The dashboard

13 views built on the saved results

Live prediction

all 25 segments, next 30 min

Risk watchlist

ranked for patrol dispatch

Congestion heatmap

where and when it runs hot

Forecast vs actual

predicted against what happened

Model performance

every method, side by side

Weather vs traffic

how rain changes flow

Safest live demo: `python demo.py` – prints the whole story in 30 seconds, nothing can fail

What I'd want you to remember



It works

8.9% error, 45% better than assuming nothing changes



The data was broken

5,203 time slots were missing and invisible — it would have corrupted everything



One column was a trap

congestion_level was secretly the vehicle count — using it gives a fake 99%



One target was impossible

and I proved it: a cheating model gained only 0.006

Thank you — questions welcome